Martin Gonzalez Rico

Excepciones

No.	Respuesta	Explicación:
1	e.Throwable	Throwable es la clase de la que heredan tanto Error como Exception. Dentro de Exception encontramos muchas excepciones que podemos manejar, mientras que Error representa problemas más graves. Aunque Object es la raíz de todas las clases Java, Throwable es la raíz específica de la jerarquía de errores y excepciones.
2	c.Checked d.Unchecked	Las excepciones se clasifican en checked y unchecked. Las checked deben capturarse con try-catch o declararse con throws, mientras que las unchecked no tienen esa obligación.
3	a.try, catch, finally, throw, throws	Estas palabras se utilizan para trabajar con excepciones en diferentes situaciones. try contiene el código que podría producir una excepción, catch permite capturarla, finally ejecuta el bloque de limpieza, throw lanza una excepción y throws declara las excepciones que un método puede propagar.
4	d.Crear y lanzar explícitamente una excepción.	La palabra throw sirve para lanzar manualmente una excepción cuando queremos indicar que ocurrió una situación incorrecta.
5	d.Se utiliza para generar una excepción manualmente.	Podemos utilizar throw para lanzar una excepción cuando una condición no se cumple.
6	c.Throws indicates the type of exception that the method does not handle and throw an exception.	La diferencia es que throws se escribe en la declaración del método o constructor para indicar qué excepciones puede propagar, mientras que throw se utiliza dentro del código para lanzar una excepción concreta.
7	b.IOException is a checked exception, and NullPointerException is an unchecked exception.	IOException es una excepción checked, por lo que debemos capturarla o declararla con throws.
8	b.ACD	El método mf() no hace nada ni lanza ninguna excepción, así que el programa entra en el try e imprime A. Como no ocurre ninguna excepción, se salta el catch. Después siempre se ejecuta finally e imprime C, y finalmente el programa continúa e imprime D. Por eso el resultado es ACD.
9	c.Se imprime "Excepción capturada: Edad no válida para votar."	El método validarEdad(16) comprueba si la edad es menor que 18 y, como lo es, lanza una IllegalArgumentException con el mensaje indicado. La excepción se captura en el catch de main, que imprime Excepción capturada: seguido del mensaje. La línea Edad válida. no se ejecuta porque la excepción se lanza antes de llegar a ella.
10	b.	La opción b es una mala práctica porque una ArrayIndexOutOfBoundsException normalmente indica que

	Checking for ArrayIndexOutOfBoundsException and ensuring that the program can recover if one occurs. d. Checking for Error and, if necessary, restarting the program to ensure that users are unaware of problems. e. Checking for ArrayIndexOutOfBoundsException when iterating through an array to determine when all elements have been visited.	el programa está accediendo a una posición incorrecta; lo correcto es validar los índices en lugar de depender de la excepción para recuperarse. La opción d también es incorrecta porque capturar un Error y reiniciar el programa para ocultar el problema puede empeorar la situación. Por ultimo, la opción e está mal porque no debemos utilizar esa excepción para saber cuándo terminamos de recorrer un arreglo; debemos comprobar el límite con una condición, como i < array.length.
11	a.IOException c.InterruptedException	Estas dos excepciones son checked, por lo que Java exige capturarlas o declararlas mediante throws cuando corresponde.
12	a.ArrayIndexOutOfBoundsException c.RuntimeException d.NumberFormatException e.IllegalArgumentException	Estas cuatro excepciones son unchecked, por lo que no estamos obligados a capturarlas.
13	a.Error: División por cero. c.Ejecución finalizada.	Primero, 10 / 0 provoca una ArithmeticException porque se trata de una división entre enteros y el divisor es cero. Por eso nunca se alcanza la instrucción que imprime Resultado: 1. El primer catch captura la excepción y muestra Error: División por cero. Después se ejecuta finally, que imprime Ejecución finalizada.. El segundo catch no se ejecuta porque el primer bloque ya capturó la excepción.
14	b.Running Problem Stopping	Primero se ejecuta el try y se imprime Running . Después se lanza una excepción con throw new Exception(), así que el resto del try se interrumpe. La excepción se captura en catch, que imprime Problem, y finalmente se ejecuta finally, que imprime Stopping. Por eso el orden es Running Problem Stopping.
15	b.Resource used Resource closed.	Resource implementa AutoCloseable y puede utilizarse dentro de un try-with-resources. Primero se ejecuta el contenido del try, que llama a resource.use() e imprime Resource used. Cuando termina el bloque, Java llama automáticamente al método close() y se imprime Resource closed. Esto evita tener que cerrar manualmente el recurso al terminar de utilizarlo.

16	a.Before if clause Exception in thread "main" java.lang.Exception at Main.doSomething (Main.java:21) at Main.main (Main.java:15	La primera línea siempre imprime Before if clause. Después depende del valor aleatorio: si Math.random() > 0.5 es verdadero, se lanza una excepción y el programa termina mostrando el mensaje de error y el stack trace; en ese caso no se imprime After if clause. Si la condición es falsa, no se lanza la excepción y se imprime After if clause.
17	a.Adding throws IOException to the main() method signature and to the doSomething() method d.Adding throws IOException to the doSomething() method signature and changing the catch argument to IOException	El problema es que doSomething() lanza una IOException, que es checked, pero no la declara con throws. Para corregirlo, primero debemos declarar throws IOException en doSomething(). Después tenemos dos alternativas: declarar también throws IOException en main, como indica la opción a, o capturar la excepción mediante catch (IOException e), como indica la opción d.
18	a.Comprobación de ArrayIndexOutOfBoundsException al iterar a través de una matriz para determinar cuándo se han visitado todos los elementos. b.Verificar errores y, si es necesario, reiniciar el programa para garantizar que los usuarios no se den cuenta de los problemas. d.Comprobar si hay ArrayIndexOutOfBoundsException y garantizar que el programa pueda recuperarse si ocurre alguna.	La opción a es una mala práctica porque intenta utilizar ArrayIndexOutOfBoundsException para detectar cuándo ya se recorrieron todos los elementos; eso debe controlarse con el índice y la longitud del arreglo. La opción b es incorrecta porque reiniciar el programa para ocultar errores graves impide solucionar el problema real. La opción d también está mal porque intenta recuperarse de una excepción de índice en lugar de prevenirla correctamente. En cambio, manejar FileNotFoundException para informar al usuario o IOException para intentar recuperarse de un problema de entrada/salida puede ser apropiado.
19	c.A java.lang.StackOverflowError is thrown.	El método go() se llama a sí mismo continuamente y no tiene una condición que detenga la recursión. Cada llamada necesita espacio en la pila de ejecución, por lo que, después de muchas llamadas, se agota ese espacio y se produce un StackOverflowError. Aunque el código utiliza x % 5 para calcular el índice del arreglo y mantenerlo entre 0 y 4, eso no evita que la recursión continúe indefinidamente.

20	a.class Test implements SampleCloseable { public void close() throws java.io.IOException { // do something } }
	c.class Test implements SampleCloseable { public void close() throws FileNotFoundException { // do something } }
	e.class Test implements SampleCloseable { public void close() { // do something }}
		La interfaz declara close() throws IOException, así que la implementación puede declarar esa misma excepción, una subclase como FileNotFoundException o no declarar ninguna excepción checked. Lo que no puede hacer es declarar una excepción checked más general, como Exception, porque ampliaría las excepciones permitidas por la interfaz. Además, una clase implementa una interfaz con implements, no con extends.

21	b.La compilación falla	El código no compila porque Exc1 es una excepción checked, pero el bloque try dentro del método a() solamente imprime un texto y no puede lanzar esa excepción. Java no permite capturar una excepción checked que ese bloque no puede producir. Aunque el método a() declare throws Exc1, eso no significa que el bloque try interno pueda lanzar automáticamente esa excepción.
22	a. Truthy.java does NOT compile.	La condición de assert debe ser una expresión de tipo boolean. En el código aparece assert (x==6) ? "x==6" : "x!=6";, cuyo operador ternario devuelve un String, no un boolean. Por eso el compilador rechaza el código antes de ejecutarlo. Si se quisiera utilizar un mensaje, la sintaxis correcta sería, por ejemplo, assert x == 6 : "x!=6";, donde los dos puntos separan la condición del mensaje.